Course Title: Cryptography and Network Security Lab	**Course Code**: BCSE309P
Faculty: Prof. Karthika V	Slot: *L15+L16*
Regno: *23BCE1099*	Name: *Arkita Barua*

Ex2b

DES

Code

Sender

```
// sender.java
import java.io.*;
import java.net.*;
import java.nio.charset.StandardCharsets;

public class des_client_sender {

    // ---------- DES TABLES ----------
    static final int[] IP = {
        58,50,42,34,26,18,10,2,60,52,44,36,28,20,12,4,
        62,54,46,38,30,22,14,6,64,56,48,40,32,24,16,8,
        57,49,41,33,25,17,9,1,59,51,43,35,27,19,11,3,
        61,53,45,37,29,21,13,5,63,55,47,39,31,23,15,7
    };
    static final int[] FP = {
        40,8,48,16,56,24,64,32,39,7,47,15,55,23,63,31,
        38,6,46,14,54,22,62,30,37,5,45,13,53,21,61,29,
        36,4,44,12,52,20,60,28,35,3,43,11,51,19,59,27,
        34,2,42,10,50,18,58,26,33,1,41,9,49,17,57,25
    };
    static final int[] E = {
        32,1,2,3,4,5,4,5,6,7,8,9,8,9,10,11,12,13,12,13,14,15,16,17,
        16,17,18,19,20,21,20,21,22,23,24,25,24,25,26,27,28,29,28,29,30,31,32,1
    };
    static final int[] P = {
        16,7,20,21,29,12,28,17,1,15,23,26,5,18,31,10,
        2,8,24,14,32,27,3,9,19,13,30,6,22,11,4,25
    };
    static final int[] PC1 = {
        57,49,41,33,25,17,9,1,58,50,42,34,26,18,10,2,59,51,43,35,27,19,11,3,
        60,52,44,36,63,55,47,39,31,23,15,7,62,54,46,38,30,22,14,6,61,53,45,37,
```

```
        29,21,13,5,28,20,12,4
    };
    static final int[] PC2 = {
        14,17,11,24,1,5,3,28,15,6,21,10,23,19,12,4,
        26,8,16,7,27,20,13,2,41,52,31,37,47,55,30,40,
        51,45,33,48,44,49,39,56,34,53,46,42,50,36,29,32
    };

    // S-boxes [8][4][16]
    static final int[][][] S = {
        {
            {14,4,13,1,2,15,11,8,3,10,6,12,5,9,0,7},
            {0,15,7,4,14,2,13,1,10,6,12,11,9,5,3,8},
            {4,1,14,8,13,6,2,11,15,12,9,7,3,10,5,0},
            {15,12,8,2,4,9,1,7,5,11,3,14,10,0,6,13}
        },{
            {15,1,8,14,6,11,3,4,9,7,2,13,12,0,5,10},
            {3,13,4,7,15,2,8,14,12,0,1,10,6,9,11,5},
            {0,14,7,11,10,4,13,1,5,8,12,6,9,3,2,15},
            {13,8,10,1,3,15,4,2,11,6,7,12,0,5,14,9}
        },{
            {10,0,9,14,6,3,15,5,1,13,12,7,11,4,2,8},
            {13,7,0,9,3,4,6,10,2,8,5,14,12,11,15,1},
            {13,6,4,9,8,15,3,0,11,1,2,12,5,10,14,7},
            {1,10,13,0,6,9,8,7,4,15,14,3,11,5,2,12}
        },{
            {7,13,14,3,0,6,9,10,1,2,8,5,11,12,4,15},
            {13,8,11,5,6,15,0,3,4,7,2,12,1,10,14,9},
            {10,6,9,0,12,11,7,13,15,1,3,14,5,2,8,4},
            {3,15,0,6,10,1,13,8,9,4,5,11,12,7,2,14}
        },{
            {2,12,4,1,7,10,11,6,8,5,3,15,13,0,14,9},
            {14,11,2,12,4,7,13,1,5,0,15,10,3,9,8,6},
            {4,2,1,11,10,13,7,8,15,9,12,5,6,3,0,14},
            {11,8,12,7,1,14,2,13,6,15,0,9,10,4,5,3}
        },{
            {12,1,10,15,9,2,6,8,0,13,3,4,14,7,5,11},
            {10,15,4,2,7,12,9,5,6,1,13,14,0,11,3,8},
            {9,14,15,5,2,8,12,3,7,0,4,10,1,13,11,6},
            {4,3,2,12,9,5,15,10,11,14,1,7,6,0,8,13}
        },{
            {4,11,2,14,15,0,8,13,3,12,9,7,5,10,6,1},
            {13,0,11,7,4,9,1,10,14,3,5,12,2,15,8,6},
            {1,4,11,13,12,3,7,14,10,15,6,8,0,5,9,2},
            {6,11,13,8,1,4,10,7,9,5,0,15,14,2,3,12}
        },{
            {13,2,8,4,6,15,11,1,10,9,3,14,5,0,12,7},
            {1,15,13,8,10,3,7,4,12,5,6,11,0,14,9,2},
```

```
            {7,11,4,1,9,12,14,2,0,6,10,13,15,3,5,8},
            {2,1,14,7,4,10,8,13,15,12,9,0,3,5,6,11}
        }
    };

    // ---------- BIT HELPERS ----------
    static long permute(long v, int[] table, int inBits, int outBits) {
        long r = 0;
        for (int i = 0; i < outBits; i++) {
            int bit = (int)((v >> (inBits - table[i])) & 1L);
            r = (r << 1) | bit;
        }
        return r;
    }
    static long rotl28(long x, int n) {
        return ((x << n) | (x >> (28 - n))) & 0x0FFFFFFFL;
    }
    static long[] makeSubkeys(long key64) {
        long k56 = permute(key64, PC1, 64, 56);
        long c = (k56 >>> 28) & 0x0FFFFFFFL;
        long d = k56 & 0x0FFFFFFFL;
        int[] shifts = {1,1,2,2,2,2,2,2,1,2,2,2,2,2,2,1};
        long[] ks = new long[16];
        for (int i = 0; i < 16; i++) {
            c = rotl28(c, shifts[i]);
            d = rotl28(d, shifts[i]);
            long cd = (c << 28) | d;
            ks[i] = permute(cd, PC2, 56, 48);
        }
        return ks;
    }
    static long feistel(long r32, long k48) {
        long e48 = permute(r32, E, 32, 48);
        long x = e48 ^ k48;
        long out = 0;
        for (int i = 0; i < 8; i++) {
            int six = (int)((x >> (42 - 6*i)) & 0x3F);
            int row = ((six >> 5) << 1) | (six & 1);
            int col = (six >> 1) & 0x0F;
            int val = S[i][row][col];
            out = (out << 4) | val;
        }
        return permute(out, P, 32, 32);
    }
    static long desEncryptBlock(long block64, long[] ks, StringBuilder dbg) {
        long ip = permute(block64, IP, 64, 64);
        dbg.append("IP: ").append(String.format("%016X", ip)).append("\n");
        long L = (ip >>> 32) & 0xFFFFFFFFL;
```

```java
        long R = ip & 0xFFFFFFFFL;
        for (int i = 0; i < 16; i++) {
            long f = feistel(R, ks[i]);
            long nL = R;
            long nR = L ^ f;
            L = nL; R = nR;
            dbg.append("Round ").append(i+1).append(": ")
               .append(String.format("%016X", (L<<32) | R)).append("\n");
        }
        long pre = (R << 32) | L; // swap
        long fp = permute(pre, FP, 64, 64);
        dbg.append("FP: ").append(String.format("%016X", fp)).append("\n\n");
        return fp;
    }
    static long pack8(String s) {
        byte[] b = s.getBytes(StandardCharsets.UTF_8);
        long v = 0;
        for (int i = 0; i < 8; i++) {
            long c = (i < b.length) ? (b[i] & 0xFFL) : 0x20; // pad with space
            v = (v << 8) | c;
        }
        return v;
    }

    // ---------- MAIN (CLIENT) ----------
    public static void main(String[] args) throws Exception {
        Socket sock = new Socket("localhost", 5000);
        BufferedReader kb = new BufferedReader(new
InputStreamReader(System.in));
        PrintWriter out = new PrintWriter(sock.getOutputStream(), true);

        System.out.print("Plaintext: ");
        String msg = kb.readLine();

        System.out.print("Key (8 chars): ");
        String key = kb.readLine();
        if (key.length() != 8) {
            System.out.println("Key must be exactly 8 characters.");
            sock.close();
            return;
        }

        long key64 = pack8(key);
        long[] ks = makeSubkeys(key64);

        byte[] data = msg.getBytes(StandardCharsets.UTF_8);
        int blocks = (data.length + 7) / 8;
```

```
        StringBuilder allDbg = new StringBuilder();
        StringBuilder ctHex = new StringBuilder();

        for (int i = 0; i < blocks; i++) {
            String chunk = msg.substring(i*8, Math.min(msg.length(),
(i+1)*8));
            long block = pack8(chunk);
            StringBuilder dbg = new StringBuilder();
            long enc = desEncryptBlock(block, ks, dbg);
            allDbg.append("BLOCK ").append(i+1).append(":\n").append(dbg);
            ctHex.append(String.format("%016X", enc));
        }

        System.out.println("\nIntermediate Results:\n" + allDbg);
        System.out.println("CipherText: " + ctHex);

        // send cipher + key (hex)
        out.println(ctHex.toString());
        out.println(String.format("%016X", key64));

        sock.close();
    }
}
```

Receiver

```
import java.io.*;
import java.net.*;

public class des_server_receiver {

    // ---------- DES TABLES ----------
    static final int[] IP = {
        58,50,42,34,26,18,10,2,60,52,44,36,28,20,12,4,
        62,54,46,38,30,22,14,6,64,56,48,40,32,24,16,8,
        57,49,41,33,25,17,9,1,59,51,43,35,27,19,11,3,
        61,53,45,37,29,21,13,5,63,55,47,39,31,23,15,7
    };

    static final int[] FP = {
        40,8,48,16,56,24,64,32,39,7,47,15,55,23,63,31,
        38,6,46,14,54,22,62,30,37,5,45,13,53,21,61,29,
        36,4,44,12,52,20,60,28,35,3,43,11,51,19,59,27,
        34,2,42,10,50,18,58,26,33,1,41,9,49,17,57,25
    };
```

```java
    static final int[] E = {
        32,1,2,3,4,5,4,5,6,7,8,9,8,9,10,11,12,13,12,13,14,15,16,17,
        16,17,18,19,20,21,20,21,22,23,24,25,24,25,26,27,28,29,28,29,30,31,32,1
    };

    static final int[] P = {
        16,7,20,21,29,12,28,17,1,15,23,26,5,18,31,10,
        2,8,24,14,32,27,3,9,19,13,30,6,22,11,4,25
    };

    static final int[] PC1 = {
        57,49,41,33,25,17,9,1,58,50,42,34,26,18,10,2,59,51,43,35,27,19,11,3,
        60,52,44,36,63,55,47,39,31,23,15,7,62,54,46,38,30,22,14,6,61,53,45,37,
        29,21,13,5,28,20,12,4
    };

    static final int[] PC2 = {
        14,17,11,24,1,5,3,28,15,6,21,10,23,19,12,4,
        26,8,16,7,27,20,13,2,41,52,31,37,47,55,30,40,
        51,45,33,48,44,49,39,56,34,53,46,42,50,36,29,32
    };

    static final int[][][] S = des_client_sender.S;

    static long permute(long v, int[] table, int inBits, int outBits) {
        long r = 0;
        for (int i = 0; i < outBits; i++) {
            int bit = (int)((v >> (inBits - table[i])) & 1L);
            r = (r << 1) | bit;
        }
        return r;
    }

    static long rotl28(long x, int n) {
        return ((x << n) | (x >> (28 - n))) & 0x0FFFFFFFL;
    }

    static long[] generateSubKeys(long key64) {
        long k56 = permute(key64, PC1, 64, 56);
        long c = (k56 >> 28) & 0x0FFFFFFFL;
        long d = k56 & 0x0FFFFFFFL;

        int[] shifts = {1,1,2,2,2,2,2,2,1,2,2,2,2,2,2,1};
        long[] keys = new long[16];

        for (int i = 0; i < 16; i++) {
            c = rotl28(c, shifts[i]);
            d = rotl28(d, shifts[i]);
```

```
            long cd = (c << 28) | d;
            keys[i] = permute(cd, PC2, 56, 48);
        }
        return keys;
    }

    static long feistel(long r, long k) {
        long e48 = permute(r, E, 32, 48);
        long x = e48 ^ k;
        long out = 0;

        for (int i = 0; i < 8; i++) {
            int six = (int)((x >> (42 - 6*i)) & 0x3F);
            int row = ((six >> 5) << 1) | (six & 1);
            int col = (six >> 1) & 0xF;
            int val = S[i][row][col];
            out = (out << 4) | val;
        }
        return permute(out, P, 32, 32);
    }

    static long decryptBlock(long block, long[] keys, StringBuilder dbg) {
        long ip = permute(block, IP, 64, 64);
        dbg.append("IP:
").append(Long.toHexString(ip).toUpperCase()).append("\n");

        long l = (ip >> 32) & 0xFFFFFFFFL;
        long r = ip & 0xFFFFFFFFL;

        for (int i = 0; i < 16; i++) {
            long f = feistel(r, keys[15 - i]); // reverse keys
            long nl = r;
            long nr = l ^ f;
            l = nl; r = nr;

            dbg.append("Round ").append(i+1).append(": ")
               .append(Long.toHexString((l<<32)|r).toUpperCase()).append("\n")
;
        }

        long pre = (r << 32) | l;
        long fp = permute(pre, FP, 64, 64);
        dbg.append("FP:
").append(Long.toHexString(fp).toUpperCase()).append("\n\n");

        return fp;
    }
```

```java
    static long hexToLong(String h) {
        return Long.parseUnsignedLong(h,16);
    }

    public static void main(String[] args) throws Exception {

        ServerSocket ss = new ServerSocket(5000);
        System.out.println("Waiting for sender...");
        Socket s = ss.accept();

        BufferedReader in = new BufferedReader(new
InputStreamReader(s.getInputStream()));

        String ctHex = in.readLine();
        String keyHex = in.readLine();

        long key64 = hexToLong(keyHex);
        long[] keys = generateSubKeys(key64);

        System.out.println("Received Cipher Text: " + ctHex);

        int blocks = ctHex.length() / 16;
        StringBuilder pt = new StringBuilder();
        StringBuilder all = new StringBuilder();

        for (int i = 0; i < blocks; i++) {
            String blk = ctHex.substring(i*16, (i+1)*16);
            long block = hexToLong(blk);

            StringBuilder dbg = new StringBuilder();
            long dec = decryptBlock(block, keys, dbg);

            all.append("BLOCK ").append(i+1).append(":\n").append(dbg);

            for (int j = 0; j < 8; j++) {
                pt.append((char)((dec >> (56 - 8*j)) & 0xFF));
            }
        }

        System.out.println("\nIntermediate Results:\n" + all);
        System.out.println("Text after decryption: " + pt.toString());

        s.close();
        ss.close();
    }
}
```

Output [screenshot]

Sender

```
PS E:\aa vit sem6\z_crypto-lab> java des_client_sender
Plaintext: Cryptography
Key (8 chars): Security

Intermediate Results:
BLOCK 1:
IP: FF9E706500FE24E3
Round 1: 00FE24E385F719C0
Round 2: 85F719C0FBA66232
Round 3: FBA6623297CB686A
Round 4: 97CB686AD15302D9
Round 5: D15302D94EF76813
Round 6: 4EF76813C1072F52
Round 7: C1072F52C75F5221
Round 8: C75F5221B4C174D9
Round 9: B4C174D966DC1598
Round 10: 66DC15983F5283C7
Round 11: 3F5283C7977CA326
Round 12: 977CA3264523348D
Round 13: 4523348D335BE510
Round 14: 335BE51056B1E6B2
Round 15: 56B1E6B26FD5A1B6
Round 16: 6FD5A1B6759DE515
FP: FD82F79073CEE43E
```

```
BLOCK 2:
IP: 0F0A000900FF0C00
Round 1: 00FF0C00CD23C10F
Round 2: CD23C10FA5C3AE52
Round 3: A5C3AE525A0F71AE
Round 4: 5A0F71AED5751AAF
Round 5: D5751AAF2B4D14B2
Round 6: 2B4D14B21CAA7236
Round 7: 1CAA72365DD8DDF9
Round 8: 5DD8DDF95F35C745
Round 9: 5F35C7458B3B46B0
Round 10: 8B3B46B0E77774E3
Round 11: E77774E3BE97319D
Round 12: BE97319D0630C04B
Round 13: 0630C04B54239148
Round 14: 54239148E9532C7C
Round 15: E9532C7C218139D9
Round 16: 218139D96F7B0CA1
FP: FB50445E1AD95223


CipherText: FD82F79073CEE43EFB50445E1AD95223
PS E:\aa vit sem6\z_crypto-lab>
```

Receiver

```
PS E:\aa vit sem6\z_crypto-lab> javac des_client_sender.java des_server_receiver.java
PS E:\aa vit sem6\z_crypto-lab> java des_server_receiver
Waiting for sender...
Received Cipher Text: FD82F79073CEE43EFB50445E1AD95223

Intermediate Results:
BLOCK 1:
IP: 759DE5156FD5A1B6
Round 1: 6FD5A1B656B1E6B2
Round 2: 56B1E6B2335BE510
Round 3: 335BE5104523348D
Round 4: 4523348D977CA326
Round 5: 977CA3263F5283C7
Round 6: 3F5283C766DC1598
Round 7: 66DC1598B4C174D9
Round 8: B4C174D9C75F5221
Round 9: C75F5221C1072F52
Round 10: C1072F524EF76813
Round 11: 4EF76813D15302D9
Round 12: D15302D997CB686A
Round 13: 97CB686AFBA66232
Round 14: FBA6623285F719C0
Round 15: 85F719C000FE24E3
Round 16: FE24E3FF9E7065
FP: 43727970746F6772
```

```
BLOCK 2:
IP: 6F7B0CA1218139D9
Round 1: 218139D9E9532C7C
Round 2: E9532C7C54239148
Round 3: 542391480630C04B
Round 4: 630C04BBE97319D
Round 5: BE97319DE77774E3
Round 6: E77774E38B3B46B0
Round 7: 8B3B46B05F35C745
Round 8: 5F35C7455DD8DDF9
Round 9: 5DD8DDF91CAA7236
Round 10: 1CAA72362B4D14B2
Round 11: 2B4D14B2D5751AAF
Round 12: D5751AAF5A0F71AE
Round 13: 5A0F71AEA5C3AE52
Round 14: A5C3AE52CD23C10F
Round 15: CD23C10F00FF0C00
Round 16: FF0C000F0A0009
FP: 6170687920202020


Text after decryption: Cryptography
PS E:\aa vit sem6\z_crypto-lab>
```